

React Native Core Components

This guide focuses on the essential core components in React Native for building mobile applications.

Prerequisites: Complete the Environment Setup Guide and create your first app.

Reference: <https://reactnative.dev/docs/getting-started>

Core Components

React Native provides built-in components that map to native platform components:

View

Container component (like `<div>` in web)

- Used to wrap and group other components
- Supports flexbox layout by default
- Can be styled with StyleSheet

Example:

```
import { View } from 'react-native';

<View style={{ padding: 20 }}>
  {/* Your content here */}
</View>
```

Text

Displays text (like `<p>` in web)

- Required for displaying any text in React Native
- Text must be wrapped in `<Text>` component
- Supports nested text with different styles

Example:

```
import { Text } from 'react-native';

<Text style={{ fontSize: 18, color: '#333' }}>
  Hello, React Native!
</Text>
```

Image

Displays images

- Can load images from local assets or remote URLs
- Requires explicit width and height (or use `resizeMode`)

Example:

```
import { Image } from 'react-native';

// Local image
<Image source={require('./assets/icon.png')} />

// Remote image
<Image
  source={{ uri: 'https://example.com/image.jpg' }}
  style={{ width: 200, height: 200 }}
/>
```

ScrollView

Scrollable container

- Allows content to scroll when it exceeds screen size
- Use for small lists or scrollable content
- For long lists, prefer `FlatList` for better performance

Example:

```
import { ScrollView, Text } from 'react-native';

<ScrollView>
  <Text>Scrollable content here...</Text>
</ScrollView>
```

TextInput

Text input field

- Allows users to enter text
- Controlled component (use `value` and `onChangeText`)
- Supports various keyboard types and input modes

Example:

```
import { useState } from 'react';
import { TextInput } from 'react-native';

function MyInput() {
```

```
const [text, setText] = useState('');

return (
  <TextInput
    value={text}
    onChangeText={setText}
    placeholder="Enter text..."
    style={{ borderWidth: 1, padding: 10 }}
  />
);
}
```

Button

Pressable button

- Simple button component with `title` and `onPress`
- Limited styling options (use `TouchableOpacity` for custom styling)

Example:

```
import { Button } from 'react-native';

<Button
  title="Click Me"
  onPress={() => alert('Button pressed!')}
/>
```

StyleSheet

Style definitions

- Used to create style objects for components
- Better performance than inline styles
- Similar to CSS but uses camelCase and JavaScript objects

Example:

```
import { StyleSheet, View, Text } from 'react-native';

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#fff',
  },
  text: {
    fontSize: 20,
  },
});
```

```
    color: '#333',
    fontWeight: 'bold',
  },
});

function MyComponent() {
  return (
    <View style={styles.container}>
      <Text style={styles.text}>Styled Text</Text>
    </View>
  );
}
```

Key Styling Notes:

- Use camelCase (e.g., `backgroundColor` not `background-color`)
- No units needed for dimensions (numbers are density-independent pixels)
- Flexbox is the default layout system

Resources

- **React Native Documentation:** <https://reactnative.dev/docs/getting-started>
- **React Native Components:** <https://reactnative.dev/docs/components-and-apis>
- **Expo Documentation:** <https://docs.expo.dev>